CHRIST (Deemed to be University), Bengaluru

Department of Computer Science

Software Requirements Specification

Course: CSC481-5 — Computer Science Project

Semester VI

ScholarLens

Offline SLM-Based Research Literature Review Agent

Sl. No.	Student Name	Register No.
1.	Aditya Mehta	2440204
2.	Jitvan Chadha	2440222
3.	Ritesh K R	2440233

Guide: Dr. New Begin

Class: 5BSC CS

June 2026

Contents

1. Project Study

1.1 Introduction

Research literature review is a fundamental step in academic and scientific work. Researchers and students routinely need to read, organise, and synthesise large volumes of published papers to understand the current state of knowledge in their field. This process is predominantly manual—time-consuming and prone to oversight, especially when dealing with dozens of papers across related sub-topics.

Recent advances in language models have enabled AI-assisted literature tools; however, these tools are cloud-dependent, raising concerns about privacy, cost, and accessibility. **ScholarLens** is proposed as a fully offline, privacy-first research assistant that leverages locally running Small Language Models (SLMs) to help researchers ingest, search, analyse, and synthesise research papers—without any data leaving the user's machine.

1.2 Existing System

Several tools exist in the research literature management space:

Tool	**Description**	**Type**
Zotero	Reference manager with PDF storage, annotation, and citation generation.	Offline/Sync
Mendeley	Reference manager with social features and PDF reader. Owned by Elsevier.	Cloud-based
Elicit	AI research assistant using LLMs to find and summarise papers.	Cloud (paid)
Semantic Scholar	Academic search engine with AI-generated TLDRs.	Cloud-based
Connected Papers	Visualises citation graphs between papers.	Cloud-based

1.3 Limitations of the Existing System

1. **Internet dependency** — Elicit, Semantic Scholar, and Connected Papers are entirely cloud-based and cannot be used offline.
2. **Privacy concerns** — Cloud tools require uploading research documents to third-party servers.
3. **Limited analytical capability** — Zotero and Mendeley are reference managers; they do not perform semantic search, summarisation, or gap analysis.

4. **Cost** — Advanced AI features in Elicit require paid subscriptions, limiting access for students.

5. **No research gap identification** — None of the existing tools automatically identify under-explored areas across a user's paper collection.

1.4 Proposed System

ScholarLens is a self-contained, offline web application that provides:

- PDF ingestion with automatic text extraction, chunking, and metadata parsing.
- Hybrid search combining semantic embeddings with BM25 keyword matching.
- SLM-powered analysis: summarisation, claim extraction, and thematic clustering.
- Automated research gap identification with suggested future directions.
- A browser-based interface requiring no additional software beyond the runtime.

1.5 Benefits of the Proposed System

1. **Complete privacy** — All processing occurs locally; no data is transmitted externally.
2. **Zero recurring cost** — Uses open-source models and tools with no subscriptions or API fees.
3. **Offline operation** — Fully functional without internet after initial setup.
4. **Comprehensive analysis** — Goes beyond reference management to deliver actionable research insights.
5. **Modular and extensible** — New capabilities can be added independently without disrupting existing functionality.

2. Literature Survey

2.1 Transformer-Based Language Models

The transformer architecture (Vaswani et al., 2017) has fundamentally changed NLP. BERT (Devlin et al., 2019) introduced bidirectional pre-training for strong performance on comprehension tasks. The LLaMA family (Touvron et al., 2023) subsequently showed that smaller open models (1B–8B parameters) can achieve competitive results on consumer hardware. Frameworks like **Ollama** now allow users to run these models locally with a simple command-line interface.

2.2 Sentence Embeddings and Semantic Search

Sentence-BERT (Reimers & Gurevych, 2019) fine-tunes BERT-like models to produce semantically meaningful sentence-level embeddings. The `all-MiniLM-L6-v2` variant offers an effective trade-off between quality and speed, producing 384-dimensional vectors suitable for cosine similarity search.

2.3 Retrieval-Augmented Generation

Lewis et al. (2020) proposed RAG, which combines a retrieval component with a generative model. Instead of relying solely on the LLM's parametric knowledge, relevant documents are retrieved and provided as context, reducing hallucination and grounding outputs in source material.

2.4 BM25 and Hybrid Retrieval

BM25 (Robertson & Zaragoza, 2009) remains a strong keyword retrieval baseline. Recent work shows that hybrid approaches—combining dense semantic embeddings with sparse BM25 scores—consistently outperform either method alone (Karpukhin et al., 2020).

2.5 Vector Databases

Vector databases such as ChromaDB and FAISS are optimised for approximate nearest-neighbour search over high-dimensional embeddings. ChromaDB is chosen for this project due to its lightweight, Python-native API and persistent file-based storage.

References:

[1] Vaswani, A. et al., “Attention Is All You Need,” *NeurIPS*, 2017.

[2] Devlin, J. et al., “BERT: Pre-training of Deep Bidirectional Transformers,” *NAACL-HLT*, 2019.

[3] Touvron, H. et al., “LLaMA: Open and Efficient Foundation Language Models,” *arXiv:2302.13971*, 2023.

[4] Reimers, N. and Gurevych, I., “Sentence-BERT,” *EMNLP*, 2019.

[5] Lewis, P. et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *NeurIPS*, 2020.

[6] Robertson, S. and Zaragoza, H., “The Probabilistic Relevance Framework: BM25 and Beyond,” *FnT in IR*, 2009.

[7] Karpukhin, V. et al., “Dense Passage Retrieval for Open-Domain QA,” *EMNLP*, 2020.

3. Requirements Specification

3.1 Functional Requirements

ID	Requirement	Description
FR-01	PDF Upload	The system shall allow users to upload research papers in PDF format via a web interface.
FR-02	Text Extraction	The system shall extract text from uploaded PDFs, handling multi-column layouts.
FR-03	Metadata Extraction	The system shall extract metadata (title, authors, abstract, year) from papers where available.
FR-04	Text Chunking	The system shall segment text into configurable chunks with overlap to preserve context.
FR-05	Embedding Generation	The system shall generate vector embeddings for each chunk using a local embedding model.
FR-06	Vector Storage	The system shall persist embeddings in a local vector database.
FR-07	Semantic Search	The system shall perform similarity search over embeddings given a natural language query.
FR-08	Keyword Search	The system shall support BM25-based keyword search.
FR-09	Hybrid Search	The system shall combine semantic and keyword search results using score fusion.
FR-10	Summarisation	The system shall generate summaries of selected papers using a local SLM.
FR-11	Claim Extraction	The system shall extract key claims and findings from papers.
FR-12	Theme Clustering	The system shall group papers by thematic similarity.
FR-13	Gap Analysis	The system shall identify under-explored areas across the user's paper collection.

ID	Requirement	Description
FR-14	Direction Suggestions	The system shall suggest potential research directions based on identified gaps.
FR-15	Library Management	The system shall allow users to browse, view, and delete ingested papers.
FR-16	Health Check	The system shall verify that all backend services are operational.

Exceptional behaviour: If the Ollama model server is not running, the system shall display a clear error message indicating the service is unavailable and disable analysis features. Corrupt or unsupported PDF files shall be rejected with an appropriate error. Search on an empty knowledge base shall return an informative empty-state message.

3.2 Non-Functional Requirements

ID	Requirement	Description
NFR-01	Offline Operation	The system shall work entirely offline after initial setup.
NFR-02	Privacy	No user data shall be sent to any external server.
NFR-03	Performance	Search queries shall return within 3 seconds for up to 100 papers.
NFR-04	Usability	The interface shall be intuitive and require no technical training.
NFR-05	Portability	The system shall run on Windows, macOS, and Linux.
NFR-06	Configurability	Key parameters (model, chunk size, top-K) shall be configurable via environment variables.

3.3 Software Requirements

Software	Specification
Operating System	Windows 10/11, macOS 12+, or Ubuntu 20.04+
Python	3.11 or higher
Node.js	18 or higher
Ollama	Latest stable release with a pulled SLM (e.g. LLaMA 3.2)
Web Browser	Chrome, Firefox, Edge, or Safari

3.4 Hardware Requirements

Component	**Minimum**	**Recommended**
Processor	Quad-core x86_64	8-core / Apple M-series
RAM	8 GB	16 GB
Storage	10 GB free	20 GB (SSD preferred)
GPU	Not required	NVIDIA 6 GB+ VRAM

4. System Models

4.1 System Architecture (Block Diagram)

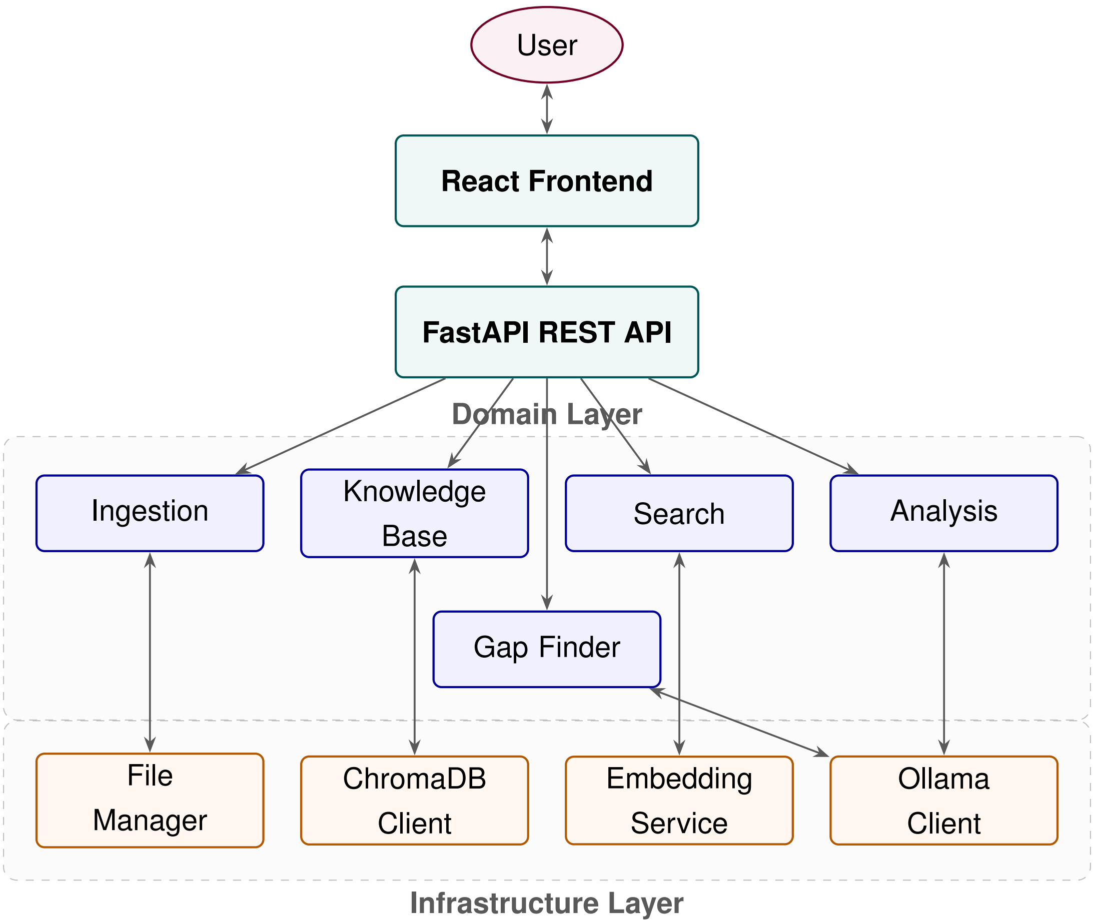

Figure 1: High-Level System Architecture of ScholarLens

4.2 Module Descriptions

4.2.1 Module 1: Document Ingestion

Handles the intake of raw PDF files and converts them into structured text.

- **PDF Parser** — Extracts text from PDF files using PyMuPDF with pdfplumber as fallback for complex layouts.
- **Text Chunker** — Splits text into fixed-size chunks (default 512 tokens, 50-token overlap) respecting sentence boundaries.
- **Metadata Extractor** — Parses PDF metadata and uses heuristics to identify title, authors, abstract, and year.

4.2.2 Module 2: Knowledge Base

Manages persistent storage and retrieval of document embeddings.

- **Embedding Service** — Generates 384-dim vectors using the `all-MiniLM-L6-v2` sentence-transformer model.
- **Repository** — CRUD operations on ChromaDB for storing and querying chunks with their embeddings.

4.2.3 Module 3: Search & Retrieval

Provides intelligent passage retrieval from the knowledge base.

- **Semantic Search** — Cosine similarity over dense embeddings.
- **Keyword Search** — BM25 ranking over the text corpus.
- **Hybrid Search** — Fuses semantic and keyword scores to return top-K results.

4.2.4 Module 4: Analysis

Uses the local SLM for higher-order text analysis.

- **Summariser** — Generates concise paper summaries by providing retrieved chunks as context to the SLM.
- **Claim Extractor** — Identifies key findings using structured prompting.
- **Theme Clusterer** — Groups papers by topic similarity.

4.2.5 Module 5: Gap Finder

Analyses the full paper collection for under-explored areas.

- **Gap Analyser** — Identifies thin, contradictory, or absent coverage areas.
- **Suggestion Engine** — Generates research direction suggestions grounded in the user's papers.

4.2.6 Module 6: Infrastructure

Adapters isolating domain logic from external services.

- **Ollama Client** — HTTP client for local SLM inference, model listing, and health checks.
- **ChromaDB Client** — Manages vector store connection and collection lifecycle.
- **File Manager** — Filesystem operations for PDF storage and directory management.

4.2.7 Module 7: API Layer

Exposes domain functionality as RESTful endpoints:

Method	**Endpoint**	**Function**
POST	/api/documents/upload	Upload and ingest a PDF
GET	/api/documents	List all papers
GET	/api/documents/{id}	Get paper details
DELETE	/api/documents/{id}	Delete a paper
POST	/api/search	Hybrid search
POST	/api/analysis/summarize	Generate summaries
POST	/api/analysis/claims	Extract claims
POST	/api/analysis/themes	Cluster by themes
POST	/api/gaps/analyze	Gap analysis
GET	/api/system/health	Health check
GET	/api/system/models	List Ollama models
GET	/api/system/stats	KB statistics

4.2.8 Module 8: Frontend

Browser-based React SPA with five views:

- **Upload Panel** — Drag-and-drop PDF upload.
- **Library** — Browse and manage ingested papers.
- **Search** — Query input with ranked passage results.
- **Analysis** — Trigger summarisation, claim extraction, and theme clustering.
- **Gap Analysis** — View research gaps and suggestions.